SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I _VINAY KUMAR.B ***(192512359)*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. AI-Powered Drone for Emergency Medicine Delivery

1.1 Explain how Greedy Best-First Search behaves and discuss its strengths and weaknesses.

Greedy Best-First Search (GBFS) is an informed search algorithm that selects the node that appears closest to the goal using a heuristic function.

The evaluation function is:

$$ f(n)=h(n) $$

Where:

- $h(n)$ = estimated distance from the current node to the destination.
- $f(n)$ = estimated priority of the node.

Working:

1. The drone identifies its starting location and destination.
2. It calculates the straight-line distance from nearby locations to the destination.
3. It selects the location with the smallest heuristic value.
4. It continues until it reaches the destination.
5. If a route becomes blocked, it updates the route and searches again.

Strengths:

- Simple and fast in many situations.
- Requires a heuristic rather than exploring every route.
- Useful when quick decisions are required.

Weaknesses:

- Ignores the cost already incurred.
- May select a short but dangerous route.
- Does not guarantee the minimum-cost route.
- Can perform poorly when weather and terrain information changes.

1.2 Explain how A* handles the same situation.

A* is an informed search algorithm that considers both the actual cost already incurred and the estimated cost remaining.

Its evaluation function is:

\[f(n)=g(n)+h(n) \]
Where:

- \(g(n)\) = actual cost from the starting point to the current node.
- \(h(n)\) = estimated cost from the current node to the destination.
- \(f(n)\) = estimated total route cost.

Working:

1. The drone identifies possible routes.
2. It calculates actual travel costs, including distance, weather, terrain, and delay.
3. It estimates the remaining travel cost using a heuristic.
4. It selects the node with the lowest \(f(n)\).
5. If a path becomes blocked, it updates the map and replans.

For example, a short route through a flooded area may have a high risk cost. A* can select a longer, safer route if the cost function represents those risks appropriately.
Advantages:

- Considers both current and future costs.
- Can find an optimal minimum-cost route under suitable conditions.
- Handles different movement costs when they are correctly modeled.

Limitations:

- May consume considerable memory and computation.
- Dynamic changes may require repeated replanning.
- Optimality depends on the cost model and heuristic.

1.3 Analyze the impact of heuristic accuracy.
A heuristic estimates the remaining distance or cost to reach the goal.

Heuristic accuracy	**Effect on GBFS**	**Effect on A***
Accurate	Usually guides the search efficiently	Usually improves search efficiency

Overestimated	May choose a misleading route	May lose its optimality guarantee
Underestimated	May still choose a poor route because actual cost is ignored	Can preserve optimality if admissibility conditions are satisfied
Outdated	May lead toward blocked paths	May select a poor route until costs are updated

Straight-line distance is useful, but it cannot fully represent flooded roads, severe weather, or dangerous terrain. A realistic cost model and updated information are essential.

1.4 Discuss how dynamic changes affect search performance.

Dynamic changes include blocked paths, weather changes, and unexpected hazards.

Their effects are:

1. Previously planned routes may become unavailable.
2. Travel costs may change.
3. The algorithm may need to search again.
4. Repeated searches may increase computation time.
5. The drone may experience delays in delivering medicine.

The drone should update its map using real-time information, remove blocked paths from consideration, and replan. Incremental search algorithms can reduce repeated computation by reusing previous search results.

1.5 Recommend the most suitable algorithm.

Recommendation: A with dynamic replanning and safety constraints.*

A* is more suitable because it considers both the actual travel cost and the estimated remaining cost. It can select a longer route when that route has a lower overall cost.

For real-time operation, the drone can use incremental or bounded search, updated weather information, and a safe fallback route. Unsafe paths should be prohibited rather than assigned only a small additional cost.

Conclusion: A* provides a better balance of cost, efficiency, and safety than Greedy Best-First Search when the environment is uncertain and the route can be updated.

2. Smart City Traffic Signal Optimization

2.1 Formulate the problem as an optimization problem and explain why traditional search is inefficient.

The objective is to optimize traffic signals across hundreds of intersections to minimize waiting time, fuel consumption, and traffic congestion.

Define the objective function as:

$$\min J=w_1W+w_2F+w_3C$$

Where:

- W = total vehicle waiting time.
- F = total fuel consumption.
- C = traffic congestion measure.
- w_1,w_2,w_3 = weights representing the importance of each objective.

Decision variables:

- Green-light duration.
- Red-light duration.
- Signal timing offsets between intersections.
- Signal phase arrangements.

Constraints:

- Minimum and maximum signal durations.
- Pedestrian crossing requirements.
- Safe intervals between conflicting traffic movements.
- Emergency vehicle priority.
- Coordination between nearby intersections.

Traditional exhaustive search is inefficient because the number of possible signal combinations increases rapidly as the number of intersections grows. Traffic conditions also change continuously, so evaluating every possible solution takes too much time.

2.2 Explain how Hill Climbing works and identify its limitations.

Hill Climbing is a local search algorithm that starts with an initial solution and repeatedly moves to a better neighboring solution.
Working:

1. Generate an initial traffic signal plan.
2. Calculate its waiting time, fuel consumption, and congestion.
3. Change one or more signal timings slightly.
4. Evaluate the new plan.
5. Accept the new plan if it improves the objective.
6. Repeat until no better neighboring solution is found or the time limit is reached.

Example: If increasing the green-light duration at a busy junction reduces total waiting time, the algorithm accepts the change.
Advantages:

- Simple to implement.
- Uses relatively little memory.
- Can improve a solution quickly.
- Useful for local timing adjustments.

Limitations:

- Can get stuck in a local optimum.
- May struggle with plateaus and ridges.
- May ignore interactions between intersections.
- Does not guarantee the global optimum.

2.3 Discuss local maxima, plateaus, and ridges.
For this minimization problem, the corresponding difficulties are local minima, plateaus, and ridges.
1. Local optimum: A solution is better than nearby alternatives, but a better solution exists elsewhere.
Example: One junction has reduced waiting time, but traffic congestion increases at neighboring junctions.
2. Plateau: Many neighboring solutions have almost the same objective value.
Example: Small changes in green-light duration do not noticeably improve traffic flow.
3. Ridge: Improvement requires a sequence of coordinated changes rather than a single adjustment.

Example: Several connected intersections must change their signal timings together to improve traffic flow.
These situations can prevent basic Hill Climbing from finding a better overall solution.

2.4 Explain how Simulated Annealing or Genetic Algorithms overcome these limitations.

A. Simulated Annealing (SA)

Simulated Annealing sometimes accepts a worse solution to escape a local optimum.
For a minimization problem, the probability of accepting a worse solution is:
\[P=e^{-\Delta J/T} \]
Where:

- \(\Delta J\) = increase in objective value.
- \(T\) = temperature parameter.

Working:

1. Start with an initial traffic signal plan.
2. Generate a neighboring plan.
3. Accept it if it improves the objective.
4. Sometimes accept a worse plan based on the temperature.
5. Gradually reduce the temperature.
6. Stop when the computation budget is exhausted.

High temperatures encourage exploration, while low temperatures encourage the algorithm to focus on better solutions.

B. Genetic Algorithm (GA)

A Genetic Algorithm uses a population of candidate solutions.
Working:

1. Generate several traffic signal plans.
2. Evaluate each plan using the objective function.
3. Select better-performing plans as parents.
4. Apply crossover to combine parts of two plans.
5. Apply mutation to introduce new timing values.
6. Repeat until a good solution is found or the time limit is reached.

Advantages:

- Explores many candidate solutions.
- Can escape some local optima.
- Handles complex combinations of signal timings.

Limitations:

- Requires parameter tuning.
- May require many evaluations.
- Does not guarantee a global optimum.

2.5 Recommend the best approach.

Recommendation: Genetic Algorithm combined with real-time traffic data and local refinement.

A Genetic Algorithm explores different signal plans and is less dependent on one initial solution. Real-time traffic data allows the system to update its plans when traffic conditions change.

Hill Climbing or Simulated Annealing can then refine a good candidate solution. All generated plans must satisfy safety and traffic signal constraints.

Conclusion: A hybrid optimization approach provides a useful balance of scalability, adaptability, and solution quality.

3. Autonomous Rover on Mars

3.1 Explain why the problem requires an online search agent instead of offline search.

An online search agent makes decisions while interacting with an environment. It does not require a complete map before starting. The Mars rover operates in unknown terrain, has limited sensing capability, and must discover hazards while exploring. Therefore, online search is suitable.

Working:

1. Observe the surrounding terrain.
2. Identify possible paths and hazards.
3. Select a safe action.
4. Move to the selected location.
5. Update the internal map using new observations.
6. Repeat until the sample is collected.

Example: If the rover discovers an unexpected crater, it updates its map and chooses a different route.

Advantages:

- Works in unknown environments.
- Adapts to new information.
- Supports continuous decision-making.
- Helps the rover explore safely.

Limitation: The rover may need to explore or backtrack because it does not know the complete environment in advance.

3.2 Analyze the challenges of partial observability and dynamic environments.

Partial observability

Partial observability means that the rover cannot observe the entire environment at once.

Challenges include:

- Hidden rocks and craters.
- Limited sensor range.
- Uncertain terrain conditions.
- Incomplete information about sample locations.

The rover must combine current sensor readings with previous observations to estimate the environment.

Dynamic environment

Environmental conditions and the rover's own situation can change during exploration.

Examples include:

- Unexpected obstacles.
- Dust storms reducing visibility.
- Unstable terrain.
- Changes in battery level or equipment condition.

The rover must update its plans whenever new information affects the estimated cost or safety of a route.

3.3 Describe how the rover builds and updates its internal model.

The rover creates an internal representation of its surroundings called a world model.

The process is:

1. Sense: Collect information using cameras and other sensors.

2. Map: Record obstacles, safe paths, and unknown locations.
3. Estimate: Calculate possible movement costs and risks.
4. Plan: Select a suitable next action.
5. Move: Travel toward the selected location.
6. Update: Modify the internal map based on new observations.

For example, a location initially marked as unknown may be marked as blocked after the rover detects a crater.

The rover repeats this cycle throughout its journey.

3.4 Compare online search with uninformed and informed search strategies.

Feature	**Uninformed search**	**Informed search**	**Online search**
Knowledge	Uses the problem definition without a heuristic	Uses a heuristic to guide search	Learns about the environment while acting
Examples	BFS, DFS	A*, Greedy Best-First Search	Online DFS and online replanning
Map	Usually assumes a known search space	Usually assumes a known search model	Can work without a complete map
Adaptability	Requires additional replanning	Can replan if information is updated	Designed for exploration and new observations
Rover suitability	Useful for small, known maps	Useful for planning over known terrain	Highly suitable for unknown terrain

An online agent can also use informed search. For example, the rover can use A* over its currently known map and replan when it discovers new terrain.

3.5 Justify the most suitable approach.

Recommendation: Online search with mapping, heuristic guidance, and safety-aware replanning.

The rover should:

- Explore unknown terrain gradually.
- Maintain an updated map.
- Use heuristic estimates to guide exploration.
- Avoid unsafe movements.
- Replan when new obstacles are detected.
- Monitor battery power and maintain a safe fallback plan.

A* can help plan routes through known areas, while online search methods help the rover explore unknown areas.

Conclusion: Online search is suitable because the rover cannot depend on a complete map. Combining exploration, informed planning, and safety checks improves adaptability and decision-making.

4. University Exam Timetable Generation Using CSP

4.1 Formulate the problem as a Constraint Satisfaction Problem (CSP).

A Constraint Satisfaction Problem (CSP) is defined by variables, domains, and constraints. The objective is to assign values to variables without violating any constraint.

It is represented as:

$$ CSP=(X,D,C) $$

Where:

- X = set of variables.
- D = domains of possible values.
- C = set of constraints.

A. Variables

For each course i, define:

- T_i: examination time slot.
- H_i: examination hall.

For example, Mathematics, Physics, and Chemistry must each receive an examination time and a hall.

B. Domains

Domains are the possible values for each variable.

Example:

- $T_i \in \{\text{Monday morning, Monday afternoon, Tuesday morning}\}$
- $H_i \in \{\text{Hall A, Hall B, Hall C}\}$

The system selects one valid time and hall for every course.

C. Constraints

Constraint	Explanation
Student conflicts	A student cannot have two exams at the same time.
Hall capacity	The selected hall must have enough seats.
Hall availability	A hall cannot host two exams simultaneously if it is a single shared space.
Subject ordering	Required prerequisite exams must be scheduled first.
Faculty availability	Exams must fit the availability of required faculty members.
Special accommodations	Accessible halls and additional time must be provided where required.
Exam duration	The time slot must accommodate the exam's duration.

The final timetable must satisfy all applicable constraints.

4.2 Explain how backtracking search works.

Backtracking search assigns values to variables one by one and reverses a choice when it causes a conflict.

Steps:

1. Start with an empty timetable.
2. Select an unscheduled course.
3. Choose a possible time slot.
4. Select an available hall.
5. Check whether the assignment satisfies all constraints.
6. If valid, continue with the next course.
7. If invalid, try another time or hall.
8. If no choice works, undo the previous assignment and backtrack.

9. Continue until a valid timetable is found or all possibilities are exhausted.

Example: If Mathematics and Physics share students, both exams cannot be scheduled in the same time slot. If a conflict occurs, the algorithm assigns Physics to another slot.

Advantages:

- Systematically explores possible timetables.
- Can find a valid solution if one exists and the search is exhaustive.
- Supports multiple constraints.

Limitation: Basic backtracking may explore too many possibilities for a large university.

4.3 Explain how constraint propagation improves efficiency.

Constraint propagation reduces the possible values for unassigned variables when a new assignment is made.

Forward checking removes domain values that conflict with the latest assignment.

Example:

1. Mathematics is assigned to Monday morning.
2. The system identifies courses that share students with Mathematics.
3. Monday morning is removed from those courses' available slots.
4. If Physics has no valid slot left, the system backtracks immediately.
5. It avoids wasting time assigning other courses before detecting the conflict.

Benefits:

- Detects conflicts early.
- Reduces the search space.
- Avoids unnecessary assignments.
- Improves search efficiency.

Other useful techniques include:

- MRV: Select the course with the fewest legal choices.

- Degree heuristic: Select the course that constrains the largest number of other courses.
- Least Constraining Value: Try the value that leaves the most choices for other courses.

4.4 Analyze real-world challenges and justify the best strategy for scalability.

Real-world scheduling involves many students, courses, faculty members, halls, and special requirements.

Challenges include:

- A large number of possible combinations.
- Student and faculty conflicts.
- Limited hall capacities.
- Last-minute changes.
- Different exam durations and accessibility requirements.

Recommendation: CSP with backtracking, forward checking, MRV, and constraint optimization.

The system should combine intelligent variable ordering, early conflict detection, and optimization of timetable quality.

Incremental rescheduling can handle changes without rebuilding the entire timetable.

Conclusion: CSP is suitable because it models timetable requirements explicitly. Constraint propagation and intelligent search improve efficiency and scalability.

5. Strategic Game AI Using Minimax and Alpha-Beta Pruning

5.1 Explain how Minimax models the problem.

Minimax is a game-search algorithm that chooses moves by considering the possible decisions of both players.

- MAX tries to maximize the evaluation score.
- MIN tries to minimize the evaluation score.

The algorithm builds a game tree containing possible moves and resulting states.

Working:

1. Generate possible moves.
2. Explore the resulting game states to a selected depth.
3. Evaluate terminal states or use an evaluation function.

4. At MAX nodes, choose the highest value.
5. At MIN nodes, choose the lowest value.
6. Select the move with the best minimax value.

The recursive formula is:

$$V(s)= \begin{cases} \max V(s') & \text{MAX's turn}\\ \min V(s') & \text{MIN's turn} \end{cases}$$

Here, s' represents a state reached through a legal move.

Example: If MAX has three possible moves with minimax values 5, 8, and 3, MAX selects the move with value 8.

Minimax assumes that the opponent will also choose strong moves. This helps the AI avoid decisions that look good initially but lead to poor outcomes later.

5.2 Analyze the computational challenges of large game trees.

The number of possible game states increases rapidly as search depth increases.

If:

nodes.

Main challenges:

- Large computation time.
- High memory requirements if many states are stored.
- Strict real-time deadlines.
- Difficulty evaluating non-terminal states.
- Changing game situations while planning.

Therefore, basic Minimax may be too expensive for complex real-time strategy games.

5.3 Explain how Alpha-Beta Pruning improves efficiency.

Alpha-Beta Pruning improves Minimax by eliminating branches that cannot affect the final decision.

It maintains two values:

Example

Suppose MAX has two choices, A and B.

- Choice A has values 5 and 6 at its MIN node. Therefore, A = 5.
- MAX now has a guaranteed value of 5, so $\alpha=5$.

- Choice B's first evaluated child has value 4.
- MIN can choose 4 or a lower value, so B cannot improve on A.
- The remaining branches under B can be skipped.

MAX chooses A with a value of 5.
Advantages:

- Reduces the number of evaluated nodes.
- Allows deeper search within the same time.
- Produces the same minimax result at the same search depth.
- Works especially well when promising moves are explored first.

With excellent move ordering, the best-case complexity can approach $O(b^{d/2})$. The worst case remains $O(b^d)$.

5.4 Design an evaluation function and justify the factors considered.

An evaluation function estimates how favorable a game state is for the AI when the search does not reach the end of the game.
One possible evaluation function is:
Where:

Factor	Meaning
M	Military strength and unit advantage
R	Available resources
B	Buildings and production capacity
O	Objective or territory control
T	Tactical advantage
D	Damage, losses, or vulnerability
$w_1,\ldots,w_6$	Importance weights for each factor

The weights are tuned according to the game's rules and tested against different opponents.
A state with strong units, sufficient resources, and control of important objectives receives a higher score. A state with damaged units and vulnerable buildings receives a lower score.

The evaluation function should also consider immediate threats, remaining game time, and victory conditions.

5.5 Recommend a strategy for balancing optimality and real-time performance.
Recommendation: Iterative deepening Minimax with Alpha-Beta Pruning and move ordering.

The system should use:

1. Iterative deepening: Search progressively deeper while time remains.
2. Alpha-Beta Pruning: Skip branches that cannot affect the result.
3. Move ordering: Explore promising moves first.
4. Evaluation function: Estimate non-terminal states.
5. Transposition table: Reuse results for previously evaluated states.
6. Time management: Stop before the deadline and return the best move found so far.

For games with simultaneous actions or very large branching factors, the AI may require a specialized search model rather than standard alternating-turn Minimax.

Conclusion:
Alpha-Beta Pruning improves search efficiency without changing the minimax result at a fixed depth. Combined with iterative deepening and a reliable evaluation function, it provides a practical balance between decision quality and real-time performance.

Overall Summary :

No.	Scenario	Recommended AI technique
1	Emergency medicine drone	A* with dynamic replanning
2	Smart city traffic optimization	Genetic Algorithm with local refinement
3	Mars exploration rover	Online search with mapping and replanning
4	University exam timetable	CSP with backtracking and forward checking
5	Strategic game AI	Minimax with Alpha-Beta Pruning and iterative deepening

Final conclusion:

Each scenario requires a different AI technique depending on the search space, uncertainty, constraints, and time available. Combining algorithms with real-time information, effective heuristics, and safety constraints can improve performance in practical applications.